Cyberscope

A TAC Security Company

# Audit Report

# SmartVault

July 2026

Repository https://github.com/infosmartvaultai/SmartVault

Commit e5ddecc247c21a56c63265d4dfd9c14f62e3859b

Audited by © cyberscope

# Table of Contents

# Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

| Severity | Likelihood / Impact of Exploitation |
| --- | --- |
| Critical | Highly Likely / High Impact |
| Medium | Less Likely / High Impact or Highly Likely/ Lower Impact |
| Minor / Informative | Unlikely / Low to no Impact |

# Review

| | |
|---|---|
| **Initial Audit** | 06 Jul 2026<br>https://github.com/cyberscope-io/audits/blob/main/1-svt/v1/audit.pdf |
| **Corrected Phase 2** | 27 Jul 2026 |

## Source Files

| | |
|---|---|
| **Repository** | https://github.com/infosmartvaultai/SmartVault |
| **Commit** | e5ddecc247c21a56c63265d4dfd9c14f62e3859b |

| Filename | SHA256 |
|---|---|
| **Vesting.sol** | b3f7b3c8b4d291d2c612814a4ac4e70305597795f66535c021b725ecefe51056 |
| **SmartVaultToken.sol** | 7c895e80d95039e1c73f697c2c3230044189663e98a7d70974cf760b4cd0d605 |
| **BotLocking.sol** | 7e11bccdc402710f00e6c5486687adab10d7173bfc4bcecc4ac5303842ba4aaf |
| **interfaces/IVesting.sol** | 35341ca57ff2ff1a05df3a08386e6ae73c86e8a0a0f1370848b1afc91c94c447 |

# Overview

The BotLocking system is a four contract suite built around a locked deposit investment product, a 7 level referral program, and a token bonus paid out through vesting. BotLocking.sol is the economic core, Vesting.sol releases the SVT bonuses over one year, IVesting.sol is the interface between the two, and SmartVaultToken.sol is the token itself. The design aims to combine fixed term locked deposits, monthly returns, multi-level referral earnings, and a vesting token bonus in one on chain product.

## BotLocking

BotLocking.sol holds the main investment lifecycle and almost all of the business logic. Users pay a fee to activate one of four plans, name a referrer on first activation, then invest. Half of every payment goes to the provider wallet and half stays in the contract's pool. Positions earn a fixed monthly return over the lock period, and users can claim rewards early or withdraw principal and reward once the lock ends. Each payout takes a fee split between the developer wallet and the referral pool, which credits uplines across seven levels anything undistributed goes to the treasury. The contract also grants SVT bonuses through the vesting contract and exposes read helpers for plans, positions, and referral data.

## Vesting

Vesting.sol is the payout layer for the SVT bonuses earned in BotLocking. Only the validator side can create schedules, and that address is set once by the owner. Each schedule unlocks in a straight line over one year, and users call release themselves to take what has unlocked. All schedules count against one global cap of ten million SVT, after which nothing further can be allocated.

## SmartVaultToken

SmartVaultToken.sol is the SVT token and the simplest piece of the system. It is a plain ERC20 with no minting after deploy, no burning, no pausing, no fees, and no blacklist. The full thirty million supply is created once and sent to a treasury address given at deploy.

## IVesting

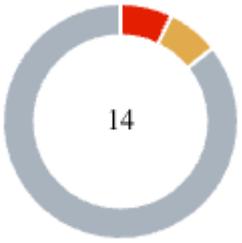
IVesting.sol is a small interface, not a deployed contract. It only lists the two functions BotLocking needs from the vesting side the one that creates a vesting schedule, and the one that reports the SVT token address. It holds no logic, no state, and no funds.

# Diagnostics

Critical Medium Minor / Informative

| Severity | Code | Description | Status |
|---|---|---|---|
| Critical | IPD | Insolvent Payout Design | Acknowledged |
| Medium | TSI | Tokens Sufficiency Insurance | Acknowledged |
| Minor / Informative | AFT | Activation Fee Truncation | Acknowledged |
| Minor / Informative | AME | Address Manipulation Exploit | Acknowledged |
| Minor / Informative | CSRL | Conditional Surplus Recovery Lock | Acknowledged |
| Minor / Informative | LRCU | Long Referral Cycles Undetected | Acknowledged |
| Minor / Informative | NSTBA | Non Standard Tokens Break Accounting | Acknowledged |
| Minor / Informative | POWD | Permissionless One Way Deposit | Acknowledged |
| Minor / Informative | RIAFP | Referrer Ignored After First Plan | Acknowledged |
| Minor / Informative | RCIL | Reward Claims Ignore Lock | Acknowledged |
| Minor / Informative | SCA | SVT Conversion Assumptions | Acknowledged |
| Minor / Informative | SDF | Sybil Downline Farming | Acknowledged |
| Minor / Informative | UDO | Unsupported Decimal Ordering | Acknowledged |
| Minor / Informative | VCFS | Vesting Cap Fails Silently | Acknowledged |

# Findings Breakdown




| | |
|---|---|
| Critical | 1 |
| Medium | 1 |
| Minor / Informative | 12 |

| Severity | Unresolved | Acknowledged | Resolved | Other |
|---|---|---|---|---|
| Critical | 0 | 1 | 0 | 0 |
| Medium | 0 | 1 | 0 | 0 |
| Minor / Informative | 0 | 12 | 0 | 0 |

# IPD - Insolvent Payout Design

| | |
|---|---|
| **Criticality** | Critical |
| **Location** | BotLocking.sol#L449,528 |
| **Status** | Acknowledged |

## Description

The `invest` function transfers half of each investment to the provider wallet and retains the remaining half in the contract, while the contract remains liable to return each participant the full principal together with the accrued reward. The amount held is therefore always less than the amount owed. no on-chain yield source  exists, so rewards and returned principal are funded from the deposits of subsequent participants, and a participant's repayment depends on later participants investing. Once new investment ceases, the balance check in `_distributeWithdraw` reverts and participants that have not yet withdrawn cannot recover their principal.

```
uint256 providerAmount = amount / 2;
contractTokenAmount += amount - providerAmount;
paymentToken.safeTransfer(provider, providerAmount);
```

```
if (contractTokenAmount < amount + reward) revert InsufficientFunds();
```

## Recommendation

The team is advised to ensure that the contract retains sufficient funds to satisfy the full principal and rewards committed to each participant, so that repayment of one participant does not depend on the deposits made by later participants and the amounts owed remain covered by funds available to the contract.

## TSI - Tokens Sufficiency Insurance

| | |
|---|---|
| **Criticality** | Medium |
| **Location** | Vesting.sol#L105,213,246<br>SmartVaultToken.sol#L8 |
| **Status** | Acknowledged |

## Description

The `SVT` tokens are not held within the Vesting contract itself. The full supply is minted to a treasury address at deploy, and the contract is designed to receive tokens from that external administrator by hand. It creates schedules and promises releases without ever checking it holds enough `SVT` to cover them. While external administration provides flexibility, it introduces a dependency on the administrator's actions and a centralization risk if the treasury underfunds the contract, release reverts and users cannot collect a bonus the system already told them they earned.

```
constructor(IERC20 _svt) Ownable(msg.sender) {
        if (address(_svt) == address(0)) revert ZeroAddress();
        svt = _svt;
    }
```

```
function release(uint256 vestingIndex) public virtual {
    address beneficiary = msg.sender;
    uint256 amount = releasable(beneficiary, vestingIndex);
    if (amount == 0) revert NothingToRelease();
    vestingSchedules[beneficiary][vestingIndex].released += amount;
    totalReleasedTokens += amount;
    emit VestingReleased(beneficiary, amount);
    svt.safeTransfer(beneficiary, amount);
}
```

## Recommendation

The team is advised to guarantee that the Vesting contract can satisfy every allocation it records, so that a release cannot revert for lack of funds once a bonus has been promised to a beneficiary. Obligations should not be created without assurance that the corresponding `SVT` will be available to the contract when a release is due.

## Team Update

The team has acknowledged that this is not a security issue and states:
The observation is right: Vesting does not check its own balance when a schedule is created. What bounds the risk is that total obligations are hard-capped. totalDistributedTokens only ever grows and is clamped at MAX_AIRDROP_POOL of 10,000,000 SVT (Vesting.sol L250-260), so lifetime releases can never exceed 10M. For production we will fund the Vesting contract with exactly 10,000,000 SVT in a single transfer, before setBotLocking() and before the first invest(). From that point the contract always holds at least what it owes, as a matter of arithmetic, and release() cannot revert for lack of funds. The funding also cannot be taken back: recoverSurplus() only opens once the pool is fully distributed, and under exact funding the recoverable surplus is zero. OpenZeppelin's VestingWallet takes the same approach and performs no balance check at schedule creation either.

svt.balanceOf(Vesting) == 10_000_000 * 10**18 (after funding, before the first invest)
Vesting.MAX_AIRDROP_POOL() == 10**25 svt.decimals() == 18

# AFT - Activation Fee Truncation

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L85,187 |
| **Status** | Acknowledged |

## Description

The constructor explicitly narrows each decimal scaled activation fee to `uint128`, although `Plan.activationFee` is declared as `uint256`. If a calculated fee exceeds the `uint128` range, the explicit conversion truncates its high-order bits before assigning the result to the wider field. Consequently, a payment token with a sufficiently large decimal count can configure activation fees substantially below their intended values. Common six and 18 decimal tokens are unaffected, but the constructor does not restrict the accepted decimal range.

```
uint256 activationFee;
...
uint256 tokenDecimals = paymentToken.decimals();
_plans[0] = Plan(
    uint128(MONTH_IN_SECONDS),
    200,
    5000,
    uint128(100 * 10 ** tokenDecimals)
);
```

## Recommendation

The team is advised to ensure that the activation fee stored for each plan always equals its intended value, so that the payment token's decimal configuration cannot cause the configured fee to differ from the amount intended.

## Team Update

The team has acknowledged that this is not a security issue and states:

The uint128 cast can only truncate once a fee value exceeds roughly 3.4e38. The first affected configuration is a payment token with 36 decimals. No such ERC-20 exists in practice, and with BSC-USD at 18 decimals the largest fee has seventeen orders of magnitude of headroom. All four stored fees are bit-exact on chain. We will drop the unnecessary cast in any future redeploy. paymentToken() == 0x55d398326f99059fF775485246999027B3197955, decimals() == 18 getPlan(0..3).activationFee == 100 / 250 / 500 / 1000 * 10**18 exactly

# AME - Address Manipulation Exploit

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L49 |
| **Status** | Acknowledged |

## Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

```
IERC20Metadata public immutable paymentToken;
```

## Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

## Team Update

The team has acknowledged that this is not a security issue and states:
The finding says functions accept external contract addresses without validation. In this contract, no runtime function does. The two contract addresses (paymentToken, svtVesting) are set once in the constructor and immutable; L49, the cited location, is a variable declaration. The only runtime address argument is the referrer in activatePlan, a wallet

address that is never called into. The constructor zero-checks all five addresses (L174-180) and probes the payment token with decimals() (L186), so deployment reverts on a non-ERC-20 address. That probe is one of the mitigations the finding itself suggests. A whitelist for constructor parameters would be maintained by the same deployer who picks the parameters, so it adds nothing. We ask for this finding to be withdrawn or restated as a note on the trusted-deployer model.

# CSRL - Conditional Surplus Recovery Lock

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | Vesting.sol#L115,116 |
| **Status** | Acknowledged |

## Description

The `recoverSurplus` function permits the owner to withdraw `SVT` held by the Vesting contract above its outstanding allocated-but-unreleased obligation. The function reverts unless `totalDistributedTokens` equals `MAX_AIRDROP_POOL`, so recovery is available only once the airdrop pool has been fully distributed. Distribution advances only as investments allocate bonuses, so a pool that settles below its maximum leaves any excess SVT in the contract with no available function able to withdraw it.

```
function recoverSurplus(address to, uint256 amount) external onlyOwner
{
    if (totalDistributedTokens != MAX_AIRDROP_POOL) revert
NotFullyDistributed();
    uint256 outstanding = totalDistributedTokens - totalReleasedTokens;
    uint256 surplus = svt.balanceOf(address(this)) < outstanding ? 0 :
svt.balanceOf(address(this)) - outstanding;
    if (amount > surplus) revert InsufficientTokens();
    svt.safeTransfer(to, amount);
    emit SurplusRecovered(to, amount);
}
```

## Recommendation

The team is advised to ensure that `SVT` held by the Vesting contract in excess of its outstanding obligations can be recovered independently of whether the airdrop pool has reached its maximum, so that excess tokens are not left unrecoverable when distribution settles below the maximum.

## Team Update

The team has acknowledged that this is not a security issue and states:
the criticized gate is the mechanism that makes the TSI funding irrevocable. Loosening it would reopen TSI.

# LRCU - Long Referral Cycles Undetected

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L560,575 |
| **Status** | Acknowledged |

## Description

The `_checkRefererCirculation` function traverses only `REF_LEVEL_COUNT` referral links, a value fixed at seven. A cycle formed among eight or more addresses is therefore not detected, because the traversal terminates before reaching the link that returns to the initial referrer. Referral distribution is likewise limited to seven levels, so an undetected cycle of this kind does not produce repeated payouts, excess distribution, or an infinite loop. The condition is informational, as it permits the referral graph to hold a cycle contrary to the acyclic structure the referral logic assumes, which could affect functionality that later depends on that assumption.

```
_participants[wallet].referrer = referrer;
_checkRefererCirculation(referrer);
```

```
function _checkRefererCirculation(address referer) internal view {
    address directReferer = referer;
    if (referer != address(0)) {
        for (uint i = 0; i < REF_LEVEL_COUNT; i++) {
            referer = _participants[referer].referrer;
            if (referer == address(0)) break;
            if (referer == directReferer) revert
ReferralCirculationDetected();
        }
    }
}
```

## Recommendation

The team is advised to ensure that the referral graph cannot contain a cycle regardless of the chain's length, so that the acyclic structure the referral distribution assumes holds for chains that exceed the number of links currently traversed.

# NSTBA - Non Standard Tokens Break Accounting

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L441,517 |
| **Status** | Acknowledged |

## Description

The `invest` and `deposit` functions credit `contractTokenAmount` with the amount the contract requests on transfer rather than the amount it actually receives. If the payment token takes a fee on transfer, the contract receives less than it records, so the recorded balance exceeds the tokens held and a later withdrawal fails. A rebasing token produces the same divergence. The payment token is fixed at deploy and is not checked for this behavior.

```
paymentToken.safeTransferFrom(msg.sender, address(this), amount);
_investments[msg.sender].push(Investment(amount, block.timestamp,
planIndex, 0, false));
_participants[msg.sender].totalLockedAmount += amount;
...
contractTokenAmount += amount - providerAmount;
```

## Recommendation

The team is advised to ensure that the recorded balance reflects the token amounts the contract actually holds, so that the accounting cannot overstate available funds and a later withdrawal cannot fail as a result of the discrepancy.

## Team Update

The team has acknowledged that this is not a security issue and states:
The audited source already states this constraint itself: the NatSpec requires a standard non-fee, non-rebasing BEP-20 and spells out what would break otherwise (L12-14, L42-48, repeated near invest and deposit). Fee and rebase behavior cannot be reliably detected at runtime, so pinning an immutable, verified standard token plus documenting the constraint is the standard way to handle it. BSC-USD's verified source has no fee or rebase logic, and

no transfer blacklist or pause either. paymentToken() ==
0x55d398326f99059fF775485246999027B3197955 (immutable)

# POWD - Permissionless One Way Deposit

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L516 |
| **Status** | Acknowledged |

## Description

The `deposit` function can be called by any address and increases `contractTokenAmount` by the transferred amount. The contract provides no mechanism to return funds added through this function, so an amount deposited in this way can only leave the contract through participant withdrawals. An address that deposits therefore contributes to the payout pool with no means of reclaiming the deposited funds.

```
function deposit(uint256 amount) public nonReentrant {
    paymentToken.safeTransferFrom(msg.sender, address(this), amount);
    contractTokenAmount += amount;
    emit Deposited(msg.sender, amount);
}
```

## Recommendation

The team is advised to ensure that contributions to the payout pool can be made only by the parties intended to provide funding, so that an address cannot irrecoverably commit funds to the pool without intending to do so.

# RIAFP - Referrer Ignored After First Plan

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L409 |
| **Status** | Acknowledged |

## Description

The `activatePlan` function records the supplied `referrer` only during a caller's first plan activation. On any subsequent activation the `referrer` argument is accepted but not applied, and the call does not revert. A caller that supplies a `referrer` on a later activation, whether to set one or to correct an existing one, receives no indication that the value had no effect.

```
bool isFirstPlanActivation = true;
for (uint256 i = 0; i < PLAN_COUNT; i++) {
    if (_participants[msg.sender].isActive[i]) {
        isFirstPlanActivation = false;
        break;
    }
}
if (isFirstPlanActivation) {
    _registerReferrer(msg.sender, referrer);
}
```

## Recommendation

The team is advised to ensure that a referrer value supplied after the first activation cannot be accepted without effect, so that a caller cannot be led to believe a referrer was recorded or updated when it was not.

# RCIL - Reward Claims Ignore Lock

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L481 |
| **Status** | Acknowledged |

## Description

The `claimRewards` function lets a user pull accrued reward without ever checking whether the lock period has passed. It only checks the pool has enough for the reward. So a user can activate a plan, invest, and then claim reward again and again from day one, while their principal stays locked and keeps earning. Because the pool is under funded, these early claims draw down a balance that is not sized to cover the obligations the contract has recorded. Whoever claims first gets paid, whoever's late gets `InsufficientFunds`.

```
function claimRewards(uint256 investmentIndex) public nonReentrant {
    if (investmentIndex >= _investments[msg.sender].length) revert
InvalidInvestmentIndex();
    Investment storage investment =
_investments[msg.sender][investmentIndex];
    if (investment.isLockedAmountWithdrawn) revert
InvestmentAlreadyWithdrawn();
    ...
    if (contractTokenAmount < reward) revert InsufficientFunds();
```

## Recommendation

The team is advised to make the treatment of rewards during the lock period deliberate and consistent with the principal withdrawal flow, and to ensure that any reward which can be claimed is one the payout pool is sized to honor, so that access to rewards does not depend on claiming ahead of other participants.

# SCA - SVT Conversion Assumptions

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L618 |
| **Status** | Acknowledged |

## Description

The SVT bonus calculation applies the plan multiplier directly to the payment-token amount and subsequently adjusts only for the difference between the tokens' decimals. Decimal normalization converts base units between the two token formats, but it does not account for their relative economic values. Consequently, the calculation implicitly treats one whole payment token as equivalent to one whole SVT. This may cause users to receive substantially more or less SVT than intended and may also accelerate exhaustion of the fixed vesting pool if SVT is undervalued by the conversion formula.

```
uint256 svtBonus =
    investmentAmount * svtBonusMultiplier / PCT_BASE;

svtBonus =
    svtBonus * 10 ** (svtDecimals - paymentTokenDecimals);
```

## Recommendation

The team is advised to ensure that the amount of `SVT` granted as a bonus corresponds to the intended value of the payment-token investment, so that the quantity of `SVT` received does not diverge from what participants are intended to be awarded.

## SDF - Sybil Downline Farming

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L504,558 |
| **Status** | Acknowledged |

## Description

The `referrer` provided at registration may be any address other than the caller, and the contract does not verify that it has activated a plan or invested. `withdrawReferralReward` allows any address holding a referral balance to withdraw it without requiring that the address is active. A single party may therefore register multiple controlled addresses, assign a controlled address as their referrer, and direct the referral fee and referral `SVT` accrued from those addresses to that referrer, so rewards accrue to positions that do not correspond to independent participants. This carries the activation fees and locked capital of the controlled addresses, a cost offset where the accrued `SVT` holds value.

```
function _registerReferrer(address wallet, address referrer) private {
    if (referrer == wallet) revert InvalidReferrer();
    _participants[wallet].referrer = referrer;
    _checkRefererCirculation(referrer);
```

```
function withdrawReferralReward() public nonReentrant {
    uint256 referralReward =
_participants[msg.sender].referralSystemReward;
    if (referralReward == 0) revert NoReferralReward();
```

## Recommendation

The team is advised to ensure that referral rewards accrue only to participants that have genuinely taken part in the system, so that rewards cannot be farmed through addresses created solely to occupy referral positions without real participation.

# UDO - Unsupported Decimal Ordering

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | BotLocking.sol#L186,619 |
| **Status** | Acknowledged |

## Description

The SVT bonus conversion assumes that SVT always has at least as many decimals as the payment token. If the payment token has more decimals, the subtraction used to calculate the scaling factor underflows and reverts under Solidity's checked arithmetic. Because _distributeSvtBonus is executed during every investment, an unsupported decimal ordering prevents all users from investing. No deployment time validation currently enforces the assumed relationship between the two tokens.

```
uint256 tokenDecimals = paymentToken.decimals();
...
uint256 svtDecimals = svt.decimals();
uint256 paymentTokenDecimals = paymentToken.decimals();
uint256 svtBonus = investmentAmount * svtBonusMultiplier / PCT_BASE;
svtBonus = svtBonus * 10 ** (svtDecimals - paymentTokenDecimals);
```

## Recommendation

The team is advised to ensure that the bonus conversion behaves correctly for any supported payment-token decimal configuration, so that the relationship between the token decimals cannot cause investments to revert.

## Team Update

The team has acknowledged that this is not a security issue and states:
The underflow needs a payment token with more decimals than SVT. SVT's 18 are hardcoded in bytecode, not a deploy parameter, so the only way to hit this is choosing a payment token above 18 decimals. BSC-USD rules that out. And because the bonus math runs inside every invest(), a wrong pairing would fail loudly on the first test investment,

never silently in production. svt.decimals() == 18 and paymentToken.decimals() == 18 on the deployed addresses

# VCFS - Vesting Cap Fails Silently

| | |
|---|---|
| **Criticality** | Minor / Informative |
| **Location** | Vesting.sol#L250,256<br>BotLocking.sol#L436,613 |
| **Status** | Acknowledged |

## Description

When the maximum airdrop pool has already been allocated, `_createVestingSchedule()` emits an `AirdropPoolExhausted` event and returns without creating a schedule or reverting.

```
if (totalDistributedTokens == MAX_AIRDROP_POOL) { emit
AirdropPoolExhausted(beneficiary, amount); return; }
```

As a result, `BotLocking.invest()` can succeed even though the user or their referrers receive no SVT allocation. `createVestingSchedule()` returns no value to the caller, and the emitted event is recorded in the transaction logs rather than returned, so `BotLocking` cannot detect from the call that the allocation was skipped and completes the investment regardless.

The same behavior occurs when the requested allocation exceeds the remaining pool. Rather than rejecting the request, the vesting contract reduces the allocation to the remaining balance and emits an `AirdropPoolPartiallyAllocated` event:

```
uint256 amountToAllocate = amount;
if (totalDistributedTokens + amount > MAX_AIRDROP_POOL) {
    amountToAllocate = MAX_AIRDROP_POOL - totalDistributedTokens;
    emit AirdropPoolPartiallyAllocated(beneficiary, amount,
amountToAllocate);
}
```

Additionally, `BotLocking` imposes no maximum investment amount. A sufficiently large investment may therefore consume the complete remaining airdrop pool through the investor's direct bonus and first-investment referral bonuses. This makes bonus availability dependent on transaction ordering and may prevent later participants from receiving their expected allocations.

## Recommendation

The team is advised to ensure that an investment cannot complete while the `SVT` allocation it promises is reduced or omitted, so that a participant is never charged for a bonus the system does not grant and the outcome of the allocation is determinable. The team is further advised to ensure that the availability of bonuses does not depend on transaction ordering, such that a single investment cannot exhaust the remaining pool to the detriment of later participants.

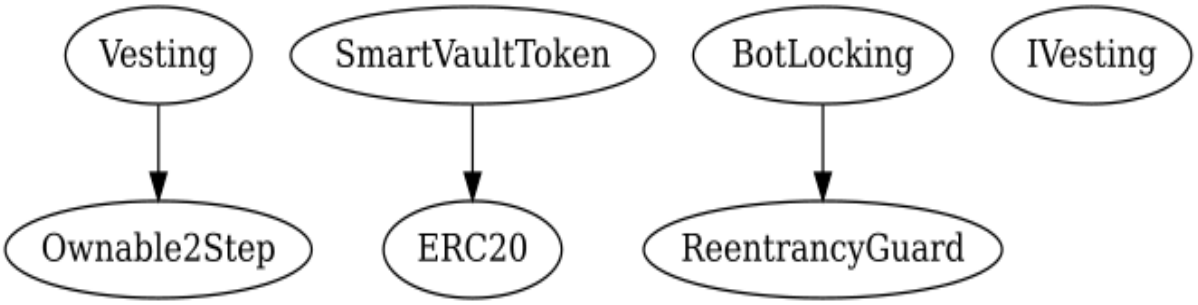
# Functions Analysis

| Contract | Type | Bases | | |
|---|---|---|---|---|
| | **Function Name** | **Visibility** | **Mutability** | **Modifiers** |
| | | | | |
| **Vesting** | Implementation | Ownable2Step | | |
| | | Public | ✓ | Ownable |
| | vestingSchedulesLength | External | | - |
| | setBotLocking | External | ✓ | onlyOwner |
| | createVestingSchedule | External | ✓ | - |
| | duration | Public | | - |
| | start | Public | | - |
| | end | Public | | - |
| | released | Public | | - |
| | releasable | Public | | - |
| | release | Public | ✓ | - |
| | vestedAmount | Public | | - |
| | _createVestingSchedule | Internal | ✓ | |
| | _vestingSchedule | Internal | | |
| | | | | |
| **SmartVaultToken** | Implementation | ERC20 | | |
| | | Public | ✓ | ERC20 |
| | | | | |

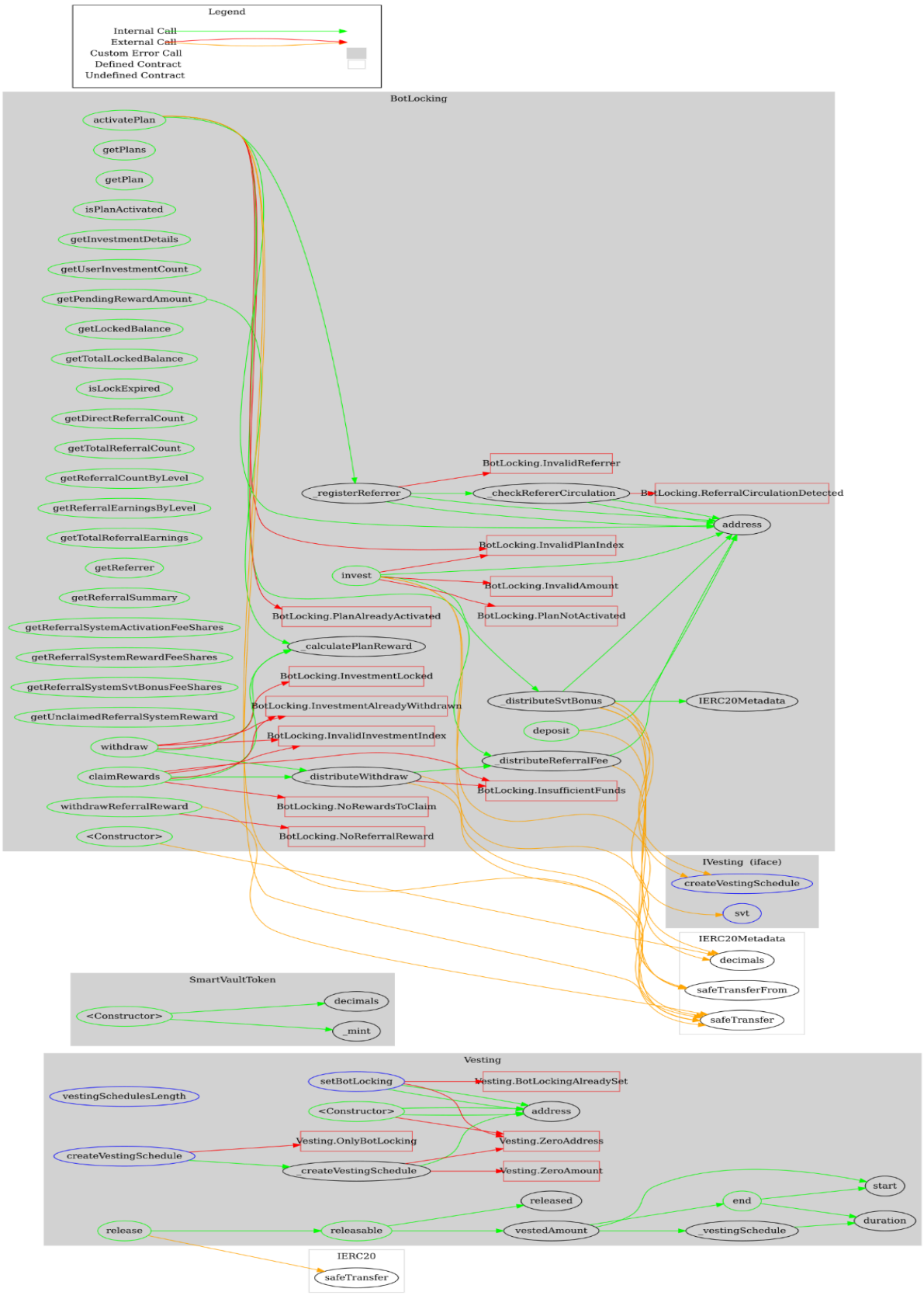
| | | | | |
|---|---|---|---|---|
| **BotLocking** | Implementation | ReentrancyGuard | | |
| | | Public | ✓ | - |
| | getPlans | Public | | - |
| | getPlan | Public | | - |
| | isPlanActivated | Public | | - |
| | getInvestmentDetails | Public | | - |
| | getUserInvestmentCount | Public | | - |
| | getPendingRewardAmount | Public | | - |
| | getLockedBalance | Public | | - |
| | getTotalLockedBalance | Public | | - |
| | isLockExpired | Public | | - |
| | getDirectReferralCount | Public | | - |
| | getTotalReferralCount | Public | | - |
| | getReferralCountByLevel | Public | | - |
| | getReferralEarningsByLevel | Public | | - |
| | getTotalReferralEarnings | Public | | - |
| | getReferrer | Public | | - |
| | getReferralSummary | Public | | - |
| | getReferralSystemActivationFeeShares | Public | | - |
| | getReferralSystemRewardFeeShares | Public | | - |
| | getReferralSystemSvtBonusFeeShares | Public | | - |
| | getUnclaimedReferralSystemReward | Public | | - |
| | activatePlan | Public | ✓ | nonReentrant |

| | | | | |
|---|---|---|---|---|
| | invest | Public | ✓ | nonReentrant |
| | withdraw | Public | ✓ | nonReentrant |
| | claimRewards | Public | ✓ | nonReentrant |
| | withdrawReferralReward | Public | ✓ | nonReentrant |
| | deposit | Public | ✓ | nonReentrant |
| | _distributeWithdraw | Private | ✓ | |
| | _calculatePlanReward | Private | | |
| | _registerReferrer | Private | ✓ | |
| | _checkRefererCirculation | Internal | | |
| | _distributeReferralFee | Private | ✓ | |
| | _distributeSvtBonus | Private | ✓ | |
| | | | | |
| **IVesting** | Interface | | | |
| | createVestingSchedule | External | ✓ | - |
| | svt | External | | - |

# Inheritance Graph



# Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
BotLocking
activatePlan
getPlans
getPlan
isPlanActivated
getInvestmentDetails
getUserInvestmentCount
getPendingRewardAmount
getLockedBalance
getTotalLockedBalance
isLockExpired
getDirectReferralCount
getTotalReferralCount
getReferralCountByLevel
getReferralEarningsByLevel
getTotalReferralEarnings
getReferrer
getReferralSummary
getReferralSystemActivationFeeShares
getReferralSystemRewardFeeShares
getReferralSystemSvtBonusFeeShares
getUnclaimedReferralSystemReward
withdraw
claimRewards
withdrawReferralReward
<Constructor>
_registerReferrer
BotLocking.InvalidReferrer
checkRefererCirculation
BotLocking.ReferralCirculationDetected
address
BotLocking.InvalidPlanIndex
invest
BotLocking.InvalidAmount
BotLocking.PlanNotActivated
BotLocking.PlanAlreadyActivated
_calculatePlanReward
BotLocking.InvestmentLocked
BotLocking.InvestmentAlreadyWithdrawn
BotLocking.InvalidInvestmentIndex
_distributeSvtBonus
IERC20Metadata
deposit
_distributeReferralFee
_distributeWithdraw
BotLocking.InsufficientFunds
BotLocking.NoRewardsToClaim
BotLocking.NoReferralReward
IVesting (iface)
createVestingSchedule
svt
IERC20Metadata
decimals
safeTransferFrom
safeTransfer
SmartVaultToken
<Constructor>
decimals
_mint
Vesting
vestingSchedulesLength
setBotLocking
Vesting.BotLockingAlreadySet
<Constructor>
address
createVestingSchedule
Vesting.OnlyBotLocking
Vesting.ZeroAddress
_createVestingSchedule
Vesting.ZeroAmount
released
start
end
release
releasable
vestedAmount
_vestingSchedule
duration
IERC20
safeTransfer

# Summary

The smartvault.ai  contracts implement a locked deposit investment suite built around fixed term plans with monthly returns, a 7 level referral program, and a token bonus layer paid out through vesting. This audit investigates security issues, business logic concerns and potential improvements.

# Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

# About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

**The Cyberscope team**

cyberscope.io